# Day 150 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 4 reference

Watch: Exponential Smoothing: Trends and Seasonality (Kevin Sheppard) | Smoothing 6: Winter's exponential smoothing (Galit Shmueli)

`day150_exponential_smoothing.py`, segment by segment, in the order `main()` uses things. The smoothing equations, the starting values, the fit from several starts, the forecasts and the interval formula are new, and are copied unchanged into today's exercise file. Day 149's Levenberg-Marquardt solver and ARIMA parts are imported from its exercise file. statsmodels is used only as a reference. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

**MATH BOX LEGEND**

**Symbols:** l = level · b = trend · s = seasonal values · $\alpha$, $\beta$, $\gamma$ = smoothing weights · $\phi$ = damping · F = a fitted model (a dictionary)

## Segment 0 — Setup

### Imports and constants

**CODE**

```
import statsmodels
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.statespace.sarimax import SARIMAX
warnings.filterwarnings("ignore")
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day147_exercise as d147
    import day149_exercise as d149
M = 12
STARTS = ((0.3, 0.05, 0.1, 0.5), (0.8, 0.5, 0.3, 0.5), (0.1, 0.01, 0.01, 0.5), (0.9, 0.9, 0.3,
    0.5))
VARIANTS = (("SES", None, None, False), ("Holt", "add", None, False), ("Holt, damped", "add",
    None, True),
            ("Holt-Winters, additive", "add", "add", False), ("Holt-Winters, multiplicative",
                "add", "mul", False),
            ("HW additive, damped", "add", "add", True), ("HW multiplicative, damped", "add",
                "mul", True))
```

- `import statsmodels` — the package itself, only for statsmodels.__version__ in the printed check.
- `from statsmodels.tsa.holtwinters import ExponentialSmoothing` — the library's smoothing models: the reference.
- `VARIANTS = (("SES", None, None, False), ...)` — a tuple of tuples: (name, trend, season, damped) for the seven variants.
- `STARTS = ((0.3, 0.05, 0.1, 0.5), ...)` — four starting points (alpha, beta, gamma, phi) for every fit.

## Segment 1 — The parts

### ets_filter: the smoothing equations

**MATH**

$$l \leftarrow \alpha(y - s) + (1 - \alpha)(l + \phi b); \quad b \leftarrow \beta(l_{new} - l) + (1 - \beta)\phi b; \quad s \leftarrow \gamma(y - l - \phi b) + (1 - \gamma)s$$

**CODE**

```
def ets_filter(y, alpha, beta, gamma, phi, l0, b0, s0, trend, season, m=M):
    l, b, s = l0, b0, np.array(s0, float).copy()
    fit = np.empty(len(y)); hist = np.empty(len(y))
    for t, yt in enumerate(y):
        st = s[t % m]
        base = l + phi * b if trend else l
        if season == "mul":
            fit[t] = base * st; ln = alpha * yt / st + (1 - alpha) * base
            s[t % m] = gamma * yt / base + (1 - gamma) * st
        elif season == "add":
            fit[t] = base + st; ln = alpha * (yt - st) + (1 - alpha) * base
            s[t % m] = gamma * (yt - base) + (1 - gamma) * st
        else:
            fit[t] = base; ln = alpha * yt + (1 - alpha) * base
        if trend:
            b = beta * (ln - l) + (1 - beta) * phi * b
        l = ln; hist[t] = s[t % m]
    return fit, l, b, s, hist
```

- `np.array(s0, float).copy()` — a fresh copy, so updating the seasonal values never changes the caller's array.
- `for t, yt in enumerate(y):` — the position and the value together.
- `s[t % m]` — % (remainder) cycles through the 12 seasonal values: month t uses slot t mod 12.
- `base = l + phi * b if trend else l` — a conditional expression: with no trend, the level alone.
- `fit[t] = base * st; ln = ...` — two statements on one line, separated by a semicolon.
- `l = ln` — the new level replaces the old only after the trend has used both.

### ets_init and fit_ets

**MATH**

*weight = 1 / (1 + $e^{-u}$), u unconstrained;  $\phi = 0.8 + 0.18 \times$ that, so $0.8 \le \phi \le 0.98$*

**CODE**

```
def ets_init(y, m=M, season="add"):
    l0 = y[:m].mean(); b0 = (y[m:2 * m].mean() - l0) / m
    return l0, b0, (y[:m] - l0 if season == "add" else y[:m] / l0)
def fit_ets(y, trend, season, damped, m=M, starts=STARTS):
    l0, b0, s0 = ets_init(y, m, season or "add")
    use = [True, trend is not None, season is not None, damped]
    def unpack(u):
        p = iter(1 / (1 + np.exp(-u)))
        a, b, g, f = (next(p) if k else d for k, d in zip(use, (0.0, 0.0, 0.0, 1.0)))
        return a, b, g, (0.8 + 0.18 * f if damped else 1.0)
    def res(u):
        return y - ets_filter(y, *unpack(u), l0, b0, s0, trend, season, m)[0]
    best = None
    for st in starts:
        p0 = np.array([v for v, k in zip(st, use) if k]); u, ssr = d149.levenberg_marquardt(res,
            np.log(p0 / (1 - p0)))
        if best is None or ssr < best[1]:
            best = (u, ssr)
    a, b, g, f = unpack(best[0]); _, l, bb, s, hist = ets_filter(y, a, b, g, f, l0, b0, s0, trend,
        season, m)
    return dict(alpha=a, beta=b, gamma=g, phi=f, sse=best[1], n=len(y), k=sum(use), level=l,
        slope=bb, season=s, hist=hist,
                trend_kind=trend, season_kind=season, init=(l0, b0, s0))
```

- `season or "add"` — or returns its first truthy operand: None becomes "add".
- `use = [True, trend is not None, ...]` — a list of True/False flags: which weights are free.
- `p = iter(1 / (1 + np.exp(-u)))` — iter turns the array into an iterator; next(p) takes the next value each time.
- `(next(p) if k else d for k, d in zip(use, ...))` — a generator, unpacked into a, b, g, f: a free weight takes the next value, a fixed one its default.
- `np.log(p0 / (1 - p0))` — the logit, the inverse of the logistic function: the unconstrained number for each start.
- `dict(alpha=a, ..., init=(l0, b0, s0))` — the fitted model as a dictionary, read later as F["alpha"].

### ets_forecast and ets_sd

**MATH**

*var(h) = $\sigma^2(1 + \Sigma_{j<h} c_j^2)$,  $c_j = \alpha(1 + j\beta) + \gamma$[j a multiple of 12]*

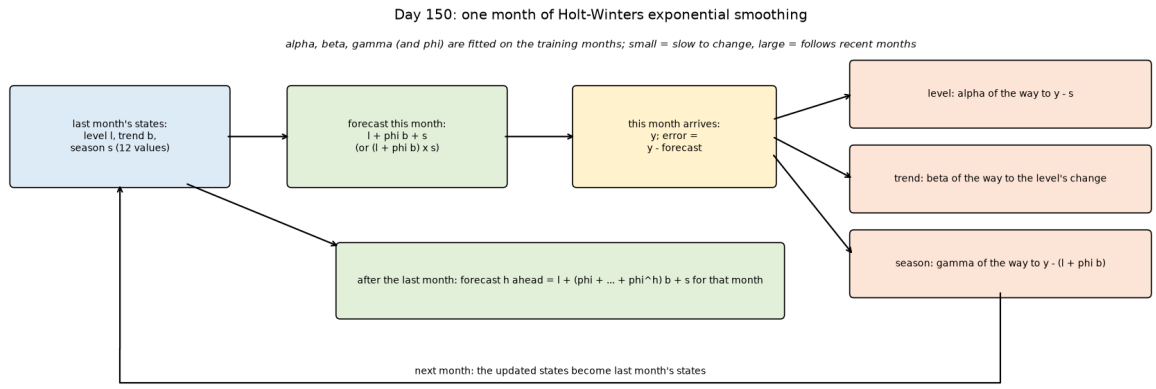
**CODE**

```
def ets_forecast(F, h, m=M):
    steps = np.cumsum(F["phi"] ** np.arange(1, h + 1)) if F["trend_kind"] else np.zeros(h)
    base = F["level"] + steps * F["slope"]
    s = F["season"][(F["n"] + np.arange(h)) % m]
    return base + s if F["season_kind"] == "add" else base * s if F["season_kind"] == "mul" else
        base
def ets_sd(F, h, m=M):
    j = np.arange(1, h)
    c = F["alpha"] * (1 + F["beta"] * j) + F["gamma"] * (j % m == 0)
    return np.sqrt(F["sse"] / F["n"] * np.r_[1.0, 1 + np.cumsum(c ** 2)])
```

- `np.cumsum(F["phi"] ** np.arange(1, h + 1))` — phi, phi + $phi^2$, ...: how many trend steps each horizon adds.

- `F["season"][(F["n"] + np.arange(h)) % m]` — fancy indexing: each future month's latest seasonal value.
- `return a if ... else b if ... else c` — a chained conditional expression: additive, multiplicative or none.
- `(j % m == 0)` — True (1) when j is a whole number of years, else False (0).
- `np.r_[1.0, 1 + np.cumsum(c ** 2)]` — the variance factors for h = 1, 2, ...: one array, one line.

**SYSTEM DIAGRAM**




*One month of Holt-Winters.*

## Segment 2 — The experiments

### airline: Day 149's model in one helper

**CODE**

```
def airline(y, h):
    w = np.diff(y[12:] - y[:-12]); th, s2, _ = d149.fit_css(w, (0, 1, 0, 1, 12)); a, b =
        d149.sarima_polys(th, (0, 1, 0, 1, 12))
    A = d149.full_ar(a); return d149.forecast(y, A, b, d149.css_residuals(w, a, b), h), np.sqrt(s2
        * np.cumsum(d149.psi_weights(A, b, h) ** 2))
```

- `np.diff(y[12:] - y[:-12])` — the seasonal difference, then the ordinary one: d1 d12 in one line.
- `d149.fit_css(w, (0, 1, 0, 1, 12))` — a part from yesterday's exercise file, called through its module name.

### demo_check: the self-check on statsmodels' forecasts

**CODE**

```
y2 = np.r_[y, ets_forecast(F, 12)]
ours = ets_filter(y2, F["alpha"], F["beta"], F["gamma"], 1.0, l0, b0, s0, "add", "add")[0][n:]
ref2 = ExponentialSmoothing(y2, trend="add", seasonal="add", seasonal_periods=M,
    initialization_method="known", initial_level=l0,
    initial_trend=b0, initial_seasonal=s0).fit(smoothing_level=F["alpha"],
        smoothing_trend=F["beta"],
    smoothing_seasonal=F["gamma"], optimized=False)
```

- `np.r_[y, ets_forecast(F, 12)]` — the training months followed by the 12 forecasts, as if they had happened.

- `ets_filter(y2, ...)[0][n:]` — the one-step forecasts for those 12 months; with zero errors they must equal the forecasts.
- `optimized=False` — statsmodels uses the given weights as they are, so both filters run with the same numbers.

### demo_choose: seven variants and a first-draft check

**CODE**

```
for name, trend, season, damped in VARIANTS:
    F = fit_ets(y, trend, season, damped); fits[name] = F
    print(f"{name:>30} {F['alpha']:7.4f} {F['beta']:7.4f} {F['gamma']:7.4f} {F['phi']:6.3f}
        {F['sse']:11.4f} {aic(F):9.2f}")
best = min(fits, key=lambda k: aic(fits[k]))
print(f"({time.perf_counter() - t0:.1f} s) AIC's choice: {best}")
three = fit_ets(y, "add", None, False, starts=STARTS[:3])
```

- `for name, trend, season, damped in VARIANTS:` — each tuple unpacked into four names.
- `min(fits, key=lambda k: aic(fits[k]))` — the dictionary key whose model has the smallest AIC.
- `starts=STARTS[:3]` — a slice of the tuple: the first three starting points only, as in the first draft.

### demo_rolling: several models per origin

**CODE**

```
for o in range(first, n - 24 + 1):
    act = yc[o:o + 24]
    for name, kind in kinds.items():
        F = fit_ets(yc[:o], "add", kind, False); f = ets_forecast(F, 24)
        E[best + " (AIC)" if name == best else name].append(f - act)
        if kind == "add":
            C["HW additive"].append(np.abs(f - act) < 1.96 * ets_sd(F, 24))
    f, sd = airline(yc[:o], 24); E["airline ARIMA, CSS"].append(f - act);
        C["airline"].append(np.abs(f - act) < 1.96 * sd)
    E["same month last year"].append(yc[o - 12:o][np.arange(24) % 12] - act)
```

- `for name, kind in kinds.items():` — both Holt-Winters kinds at every origin.
- `E[best + " (AIC)" if name == best else name]` — a conditional expression chooses the dictionary key.
- `f, sd = airline(yc[:o], 24)` — the helper returns two arrays, unpacked at once.

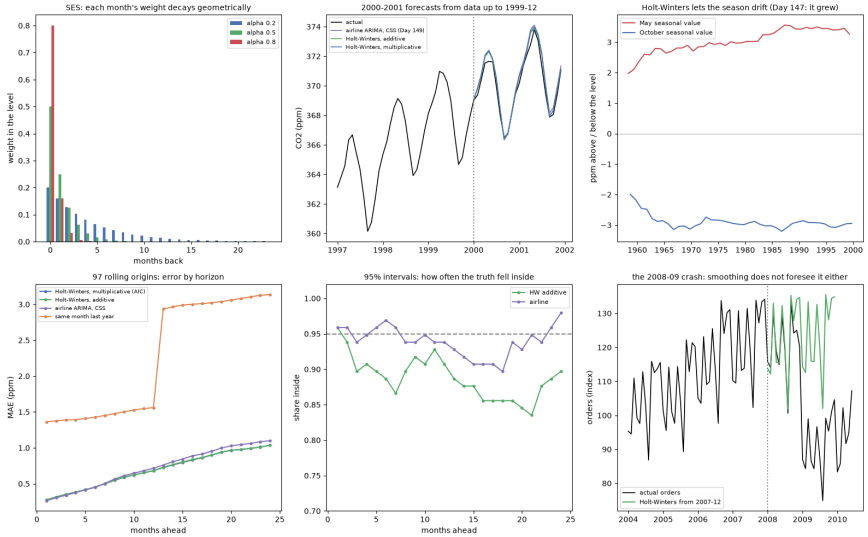
### demo_orders: a small function made on the spot

**CODE**

```
act = le[o:o + 12]; pe = lambda f: np.abs(np.exp(f - act) - 1)
E["Holt-Winters, additive"].append(pe(ets_forecast(fit_ets(le[:o], "add", "add", False), 12)))
E["airline ARIMA, CSS"].append(pe(airline(le[:o], 12)[0])); E["same month last
    year"].append(pe(le[o - 12:o]))
```

- `pe = lambda f: ...` — a one-line function; it uses this origin's act, because it is made inside the loop and used at once.
- `airline(le[:o], 12)[0]` — [0] keeps the forecast and drops the standard deviation.

**REAL GRAPH**




*Real graph from the script.*

## Quick Syntax Glossary — new or reinforced constructs

- `x.copy()` — a separate array, so changes do not reach the original.
- `t % m` — the remainder: cycles 0, 1, ..., m – 1.
- `a or b` — a if a is truthy, else b.
- `iter(x); next(p)` — step through values one at a time.
- `dict(k=v, ...)` — a dictionary from keyword arguments.
- `a if c1 else b if c2 else c` — a chained conditional expression.
- `statsmodels.__version__` — the installed version, as a string.

*End of Day 150 syntax reference. For the experiments and what they showed, see the Day 150 complete guide.*